UNIVERSIDADE ESTADUAL DA PARAÍBA
CENTRO DE CIÊNCIA E TECNOLOGIA - CCT
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

GUILHERME SILVA HENRIQUES DE SOUZA
KAYLLANE ARAÚJO DOS SANTOS
LUIS GUSTAVO SOUZA LOURENÇO

DOCENTE: FÁBIO LUIZ LEITE JÚNIOR

Relatório submetido no curso de Ciência da Computação da Universidade Estadual da Paraíba, na disciplina de Técnicas de Análise de Algoritmos, correspondente ao período de 2025.2.

CAMPINA GRANDE
2026

SUMÁRIO

1. INTRODUÇÃO.....3
1.1. Contextualização do Problema.....3
1.2. Objetivos..... 3
1.2.1. Geral.....3
1.2.2. Específico.....3
2. METODOLOGIA E IMPLEMENTAÇÃO..... 4
2.1. Estrutura do Sistema Desenvolvido..... 4
2.2. Formato de Dados de Entrada e Saída.....4
2.3. Métricas de Avaliação..... 5
3. ALGORITMOS IMPLEMENTADOS.....6
3.1. Backtracking.....6
3.2. Branch and Bound.....7
4. EXPERIMENTOS E RESULTADOS..... 8
5. ANÁLISE E DISCUSSÃO.....10
6. CONCLUSÃO..... 12
7. REFERÊNCIAS..... 13

1. INTRODUÇÃO

1.1. Contextualização do Problema

O presente trabalho tem como objetivo apresentar o estudo das N-RAINHAS, em que as rainhas são posicionadas em um tabuleiro NxN de tal forma que duas rainhas não podem se cruzar diagonalmente, nem em linhas e colunas. Este problema é um clássico da inteligência artificial, frequentemente utilizado para estudar busca, backtracking e otimização.

1.2. Objetivos

1.2.1. Geral

Analisar e compreender o problema das N-RAINHAS, explorando suas restrições e métodos de resolução por meio de técnicas computacionais utilizadas na Ciência da Computação e na Inteligência Artificial.

1.2.2. Específico

- Implementar soluções para o problema das N-RAINHAS utilizando diferentes estratégias de projeto de algoritmos:
  - Backtracking
  - Branch and Bound
- Realizar uma comparação formal entre as técnicas implementadas, considerando diferentes critérios de avaliação.
- Analisar a complexidade assintótica teórica dos algoritmos utilizados.
- Medir e comparar tempo de execução empírico das soluções implementadas.
- Comparar a qualidade das soluções obtidas, verificando se são ótimas ou aproximadas.
- Avaliar a escalabilidade dos algoritmos, observando seu comportamento para diferentes tamanhos de tabuleiro.
- Analisar as podas realizadas no espaço de busca, identificando como cada técnica reduz o número de soluções exploradas.
- Apresentar uma justificativa técnica explicando de que forma cada estratégia contribui para otimizar a busca no espaço de possíveis soluções.

2. METODOLOGIA E IMPLEMENTAÇÃO

2.1. Estrutura do Sistema Desenvolvido

O sistema desenvolvido foi implementado na linguagem Java, com o objetivo de comparar diferentes técnicas de algoritmos aplicados ao problema das N-RAINHAS. O programa foi estruturado de forma modular, seguindo princípios de orientação a objetos. A execução é iniciada por uma classe principal chamada **NQueens**, responsável apenas pela interação com o usuário e pelo controle do fluxo geral da aplicação. A lógica de resolução do problema foi abstraída em uma classe base chamada **NQueensSolver**, que define a estrutura comum para os algoritmos, incluindo métricas de desempenho e métodos auxiliares para exibição dos resultados.

As estratégias de resolução foram implementadas em classes específicas que estendem a classe base:

- **BacktrackingSolver**, que utiliza a abordagem clássica de backtracking;
- **BranchAndBoundSolver**, que aplica a técnica de branch and bound com otimizações no controle de conflitos

Dessa forma, o sistema permite:

- Receber o valor de N informado pelo usuário por meio da classe principal;
- Selecionar dinamicamente o algoritmo a ser executado;
- Executar cada estratégia de forma independente;
- Medir o tempo de execução de cada algoritmo;
- Contabilizar o número de estados explorados e backtracks realizados;
- Exibir os resultados e o tabuleiro solução de forma organizada.

Essa organização torna o código mais modular, reutilizável e fácil de expandir, permitindo a adição de novos algoritmos sem impactar a estrutura principal da aplicação.

2.2. Formato de Dados de Entrada e Saída

O programa recebe como entrada um valor inteiro N, que representa o tamanho do tabuleiro e a quantidade de rainhas a serem posicionadas no problema das N-Rainhas. Esse valor é informado pelo usuário durante a execução do programa. Caso N seja maior que zero, o algoritmo é executado; caso N seja igual a zero, o programa é encerrado. Após cada execução, o usuário pode escolher testar outro valor de N.

Como saída, o programa executa duas estratégias de resolução: Backtracking e Branch and Bound. Para cada uma delas, são exibidas a solução encontrada, a representação do tabuleiro, o tempo de execução, o número de nós explorados e a quantidade de backtracks, que indica quantas vezes o algoritmo precisou voltar para tentar outras possibilidades. Essas informações permitem comparar o desempenho das duas técnicas.

2.3. Métricas de Avaliação

Para realizar a comparação entre as diferentes técnicas de projeto de algoritmos, foram utilizadas algumas métricas de avaliação durante a execução do programa. A primeira métrica analisada foi o tempo de execução, que foi medido utilizando o método System.nanoTime(). Esse método permite registrar com alta precisão o instante de início e fim da execução do algoritmo, possibilitando calcular o tempo necessário para encontrar a solução. Posteriormente, esse valor é convertido para milissegundos (ms) para facilitar a análise e comparação entre os resultados.

Outra métrica utilizada foi a quantidade de estados explorados, representada no código pela variável nodesExplored. Essa variável contabiliza quantas posições do espaço de busca foram analisadas pelos algoritmos durante o processo de resolução do problema. A partir dessa métrica, é possível avaliar a eficiência da busca, a quantidade de tentativas realizadas e o impacto das podas no espaço de busca, especialmente no caso do algoritmo Branch and Bound, que utiliza técnicas para reduzir o número de estados que precisam ser explorados. Além disso, também foi observada a quantidade de backtracks, que indica quantas vezes o algoritmo precisou retornar a um

estado anterior para tentar uma nova possibilidade de posicionamento das rainhas.

3. ALGORITMOS IMPLEMENTADOS

3.1. Backtracking

O backtracking é uma técnica baseada em busca recursiva no espaço de soluções. Esse método constrói soluções de forma incremental, testando possíveis configurações e retrocedendo (backtrack) sempre que identifica que uma determinada escolha levará a uma solução inválida.

No contexto das N-RAINHAS, o algoritmo posiciona uma rainha em cada linha do tabuleiro, verificando se a posição escolhida não entra em conflito com a posição de rainhas já posicionadas anteriormente. Caso seja detectado algum conflito, o algoritmo descarta essa posição e tenta outra possibilidade.

Essa abordagem garante que todas as combinações válidas sejam exploradas de forma sistemática, permitindo encontrar uma solução correta para o problema. No entanto, o espaço de busca cresce rapidamente com o aumento de N, o que pode resultar em um alto custo computacional em termos de tempo de execução para valores maiores de entrada.

No sistema desenvolvido, o backtracking foi implementado utilizando recursão, onde cada chamada da função tenta posicionar uma rainha em uma nova linha do tabuleiro e verifica se a posição é segura antes de prosseguir para a próxima etapa da busca.

Em relação à corretude, o algoritmo garante que nenhuma solução inválida seja aceita, pois antes de posicionar cada rainha é realizada uma verificação de segurança que impede conflitos de coluna ou diagonal com rainhas já posicionadas. Dessa forma, quando o algoritmo alcança a última linha do tabuleiro, tem-se uma configuração válida para o problema.

Quanto ao desempenho, o algoritmo pode apresentar crescimento exponencial no número de estados explorados, uma vez que diversas combinações precisam ser testadas até que uma solução válida seja encontrada. Assim, à medida que o valor de N aumenta, cresce também a

quantidade de tentativas e retrocessos realizados pelo algoritmo, impactando diretamente no tempo de execução.

3.2. Branch and Bound

O Branch and Bound é uma técnica utilizada para resolver problemas de busca e otimização, baseada na exploração sistemática do espaço de soluções. A ideia principal consiste em dividir o problema em subproblemas menores (*branch*) e eliminar antecipadamente aqueles que não podem levar a uma solução válida (*bound*), reduzindo assim o número de possibilidades analisadas.

No problema das N-Rainhas, essa técnica funciona de forma semelhante ao backtracking, porém com um mecanismo mais eficiente de verificação de conflitos. Em vez de verificar repetidamente todas as rainhas já posicionadas, o algoritmo mantém estruturas auxiliares que indicam se determinadas colunas ou diagonais já estão ocupadas por outras rainhas.

No sistema desenvolvido, foram utilizados três vetores booleanos para realizar esse controle:

- cols: indica se uma coluna do tabuleiro já está ocupada por uma rainha.
- diag1: representa as diagonais principais do tabuleiro.
- diag2: representa as diagonais secundárias do tabuleiro.

Com essas estruturas, o algoritmo consegue verificar rapidamente se uma posição é válida antes de posicionar uma nova rainha. Caso uma coluna ou diagonal já esteja ocupada, aquela ramificação da busca é imediatamente descartada, caracterizando a poda no espaço de busca.

O algoritmo então prossegue posicionando a rainha apenas nas posições consideradas seguras. Caso não exista posição válida em determinada linha, o algoritmo retorna para a linha anterior e tenta uma nova possibilidade.

Em relação à corretude, o algoritmo garante que nenhuma configuração inválida seja aceita, pois antes de posicionar cada rainha é verificado se a

coluna e as diagonais correspondentes estão livres. Assim, quando o algoritmo consegue posicionar todas as rainhas no tabuleiro, tem-se uma solução válida para o problema.

Quanto ao desempenho, o uso das estruturas auxiliares permite reduzir o custo das verificações de conflito, tornando o processo de busca mais eficiente. Além disso, a poda antecipada de configurações inválidas diminui a quantidade de estados explorados, o que tende a reduzir o tempo de execução quando comparado a abordagens que realizam verificações repetitivas. Dessa forma, o Branch and Bound pode apresentar melhor desempenho na exploração do espaço de soluções, principalmente para valores maiores de N.

4. EXPERIMENTOS E RESULTADOS

Para avaliar o comportamento dos algoritmos Backtracking e Branch and Bound na resolução do problema das N-Rainhas, foram realizados experimentos utilizando diferentes valores de entrada para N, representando o tamanho do tabuleiro. O objetivo desses testes foi analisar como cada técnica se comporta conforme o tamanho do problema aumenta.

Durante os experimentos, foram coletadas principalmente duas métricas: nós explorados e número de backtracks. A métrica de nós explorados representa quantos estados da árvore de busca foram analisados pelo algoritmo até encontrar uma solução válida. Já o número de backtracks indica quantas vezes o algoritmo precisou desfazer uma escolha e retornar para um estado anterior da árvore de busca.

Os testes foram executados utilizando os valores N = 4, N = 8, N = 16 e N = 24, permitindo observar o crescimento do espaço de busca conforme o tamanho do tabuleiro aumenta.

Entrada para N = 4

Backtracking	Branch and Bound
Solução encontrada: 2 4 1 3 Tempo: 0,1037 ms Nós explorados: 26 Backtracks: 4	Solução encontrada: 2 4 1 3 Tempo: 0,1802 ms Nós explorados: 26 Backtracks: 22

Entrada para N = 8

Backtracking	Branch and Bound
Solução encontrada: 1 5 8 6 3 7 2 4 Tempo: 0,5949 ms Nós explorados: 876 Backtracks: 105	Solução encontrada: 1 5 8 6 3 7 2 4 Tempo: 0,2183 ms Nós explorados: 876 Backtracks: 868

Entrada para N = 16

Backtracking	Branch and Bound
Solução encontrada: 1 3 5 2 13 9 14 12 15 6 16 7 4 11 8 10 Tempo: 8,4725 ms Nós explorados: 160712 Backtracks: 10036	Solução encontrada: 1 3 5 2 13 9 14 12 15 6 16 7 4 11 8 10 Tempo: 3,8935 ms Nós explorados: 160712 Backtracks: 160696

Entrada para N = 24

Backtracking	Branch and Bound
Solução encontrada: 1 3 5 2 4 9 11 14 18 22 19 23 20 24 10 21 6 8 12 16 13 7 17 15 Tempo: 162,3061 ms Nós explorados: 9878316 Backtracks: 411584	Solução encontrada: 1 3 5 2 4 9 11 14 18 22 19 23 20 24 10 21 6 8 12 16 13 7 17 15 Tempo: 97,4262 ms Nós explorados: 9878316 Backtracks: 9878292

5. ANÁLISE E DISCUSSÃO

A partir dos experimentos realizados com diferentes valores de N, foi possível observar o comportamento dos algoritmos Backtracking e Branch and Bound na resolução do problema das N-Rainhas. O aumento do tamanho do tabuleiro provocou um crescimento significativo no número de estados analisados pelos algoritmos, evidenciando o caráter combinatório do problema.

Inicialmente, para valores menores de N, como N=4 e N=8, o número de nós explorados foi relativamente pequeno. Nesses casos, ambos os algoritmos apresentaram praticamente o mesmo comportamento, pois o espaço de busca ainda é reduzido e a solução pode ser encontrada rapidamente dentro da árvore de estados.

Entretanto, conforme o valor de N aumenta, o crescimento do espaço de busca torna-se bastante evidente. Para N=16, por exemplo, já é possível observar um aumento expressivo no número de nós explorados, ultrapassando centenas de milhares de estados analisados. Esse crescimento se torna ainda mais evidente para N=24, onde o número de nós explorados alcança milhões. Esse comportamento ocorre porque o problema das N-Rainhas possui uma complexidade combinatória elevada, na qual o número de possíveis configurações cresce rapidamente à medida que o tamanho do tabuleiro aumenta.

Outro aspecto observado nos experimentos foi que ambos os algoritmos exploraram a mesma quantidade de nós da árvore de busca em todos os testes realizados. Isso ocorre porque, na implementação utilizada, ambos percorrem a árvore de estados de maneira semelhante até encontrar a primeira solução válida. Dessa forma, mesmo utilizando estruturas auxiliares para verificar conflitos de forma mais rápida, o algoritmo Branch and Bound não reduziu a quantidade total de nós explorados nesta implementação específica.

Por outro lado, foi possível observar diferenças no número de backtracks registrados por cada algoritmo. O Backtracking apresenta uma quantidade menor de retrocessos, pois contabiliza apenas quando uma escolha precisa ser desfeita após uma tentativa de posicionamento. Já no Branch and Bound, diversas tentativas de posicionamento são descartadas imediatamente devido às restrições de coluna e

diagonal, o que faz com que esses descartes sejam contabilizados como retrocessos, resultando em valores maiores dessa métrica.

De modo geral, os experimentos demonstram que o crescimento do espaço de busca é o principal fator que influencia o desempenho dos algoritmos para o problema das N-Rainhas. Mesmo com estratégias de poda, o número de estados possíveis aumenta rapidamente com o valor de N, o que torna a resolução do problema progressivamente mais complexa.

Assim, a análise dos resultados confirma que ambos os algoritmos são capazes de encontrar soluções válidas para o problema, porém o custo de exploração da árvore de estados cresce de forma significativa à medida que o tamanho do problema aumenta.

6. CONCLUSÃO

Neste trabalho foram estudadas e implementadas duas técnicas de projeto de algoritmos aplicadas ao problema das N-Rainhas: Backtracking e Branch and Bound. A partir dos experimentos realizados com diferentes valores de N, foi possível observar como cada abordagem se comporta em relação à exploração do espaço de busca e ao desempenho do algoritmo.

O Backtracking é uma técnica normalmente utilizada para resolver problemas de decisão, realizando uma busca em profundidade na árvore de estados. Nesse método, o algoritmo tenta construir a solução passo a passo e, quando encontra um conflito, volta para o estado anterior para testar outra possibilidade. Dessa forma, a busca continua até encontrar uma configuração válida, momento em que o algoritmo pode encerrar a execução. Nos experimentos realizados, foi possível observar que, conforme N aumenta, o número de estados explorados e de retrocessos também cresce bastante, mostrando o aumento da complexidade do problema.

Já o Branch and Bound é mais utilizado em problemas de otimização e possui a vantagem de eliminar antecipadamente caminhos que não podem levar a uma solução válida. No algoritmo implementado, isso foi feito utilizando estruturas auxiliares que indicam se determinadas colunas ou diagonais já estão ocupadas. Com isso, o algoritmo consegue identificar rapidamente posições inválidas e descartar essas ramificações da busca, tornando o processo mais eficiente.

A partir dos resultados obtidos, foi possível perceber que ambas as técnicas conseguem encontrar soluções corretas para o problema das N-Rainhas. No entanto, o Branch and Bound tende a apresentar melhor desempenho em alguns casos, principalmente porque reduz o custo das verificações e descarta mais rapidamente configurações inválidas. Assim, o estudo permitiu entender melhor como diferentes técnicas de projeto de algoritmos influenciam na forma como o espaço de soluções é explorado e no desempenho da solução conforme o tamanho da entrada aumenta.

7. REFERÊNCIAS

DILLIHANGRAE. *N-Queen Problem (Backtracking Algorithm).* Medium, 16 ago. 2024. Disponível em: https://medium.com/@dillihangrae/n-queen-problem-backtracking-algorithm-55ef786c3116. Acesso em: 16 mar. 2026.

IACONO, Andrea. *Backtracking Explained.* Medium. Disponível em: https://medium.com/@andreaiacono/backtracking-explained-7450d6ef9e1a. Acesso em: 16 mar. 2026.

ANDRETTA, Marcelo. *Técnicas de Backtracking.* Instituto de Ciências Matemáticas e de Computação – Universidade de São Paulo (USP). Disponível em: https://sites.icmc.usp.br/andretta/ensino/aulas/sme0510-2-19/aula10.pdf. Acesso em: 16 mar. 2026.

GEEKSFORGEEKS. *Difference between Backtracking and Branch and Bound Technique.* Disponível em: https://www.geeksforgeeks.org/dsa/difference-between-backtracking-and-branch-n-bound-technique/. Acesso em: 16 mar. 2026.